

Specification 2 for the Cleaning Buddy App

This Cleaning Buddy app will notify users of chores that need to be completed on their cleaning day. When the Cleaning Buddy app is first setup, the user will pick the day they want to clean on, set their neat freak score, select their dwelling type, and prioritize which rooms to clean first. The app will be setup on a week by week basis, so as to minimize distractions from dating and college work.

When the user opens an account, they will take a cleanliness quiz that will give them a neat freak rating out of the 5 ratings listed below:

1. 0-20 = Minimalist Maintainer
2. 21-40 = Casual Cleaner
3. 41-60 = Routine Ready
4. 61-80 = Neat Freak
5. 81-100 = Spotless Specialist

The questions that will be asked to determine your neat freak rating are as follows:

How often do you want to vacuum or sweep the floors?

How often do you want to clean the bathroom?

How often do you want to wipe down the kitchen counters/stove?

How often do you want to do the dishes?

How often do you want to take the trash/recycling out?

How quickly do clutter bother you? (i.e.: Clothes on the floor, stuff on the counters, trash around)

How often do you?? Want to change the bedding?

How often do you want to do laundry?

Based on the above questions, your Neat Freak Title will be assigned to you.

You will be able to prioritize which top 2 cleanings mean the most to you during initial account setup.

Other priorities include how late the cleaning is and will be shown in red

Can only change dates on what matters most by one day or one week. The client wants to be able to type in date at this point if necessary.

The app will give in-app popup reminders on what needs to be completed today and push notification on what needs to be cleaned each week, only once (the client doesn't want his calendar inundated with notifications). Reminders and Push Notifications will be setup during the user's initial setup. Each user customizes their cleaning list based on their dwelling type. Under each room in the dwelling, there will be a checklist of cleaning tasks that need to be completed. All have to be completed for you to get credit for cleaning the room.

Each Room has a designated number of tasks listed below, each room needs to have a checklist with boxes that can be checked off when cleaned with the items listed below for each room:

The Kitchen will have 11 items to be completed

- Wipe down the Counter Tops
- Clean out the Refrigerator
- Wipe down the Stove
- Clean the oven
- Wash dishes
- Wipe down the walls
- Sweep the floor
- Mop the floors
- Clean the microwave
- Clean the cabinet out underneath the sink
- Take the trash/recyclables out

The Bathroom will have 7 items to be completed

- Clean the toilet
- Clean the bathtub/shower
- Sweep the floor
- Mop the floor
- Clean the mirror
- Clean the sink and countertop
- Clean the cabinet out underneath the sink

The Bedroom will have 8 items to be completed

- Make the bed
- Vacuum
- Clean the window seals
- Clean the baseboards
- Remove dishes and debris from bedroom
- Dust
- Clean off dresser
- Clean the windowsill

The Living Room will have 5 item to be completed

- Vacuum the floor
- Wipe down the furniture with pledge
- Dust
- Discard debris from the living room (shoes, clothes, video games)
- Clean the windowsill

The Laundry will have 5 Items to be completed

- Separate the clothes
- Wash whites
- Wash colors

- Fold clothes
- Put Clothes away

You will be able to postpone cleanings either by one day or by one week. The client changed this and wants to be able to input a dates here, so there will be 3 options, one day, one week, or the user can input a date. Calendar view may work here to click on a day on the calendar.

“This time tomorrow”

“This time next week”

Date Input from Calendar View.

Based on your Cleanliness Health score you receive each week, you will be able to share your score with others by copying and pasting the score to your clipboard and using it when needed (at the user’s discretion.)

There will be an analytics page explaining what tasks were completed and what needs to be worked on more.

There will be a cleaning tips page with the following tips, use best practices from the internet on cleaning tips:

- Dust before sweeping
- Always clean top to bottom
- Always dry clean before wet clean
- Empty your vacuum
- Have towels on hand
- Replenish your cleaning supplies

User Constituencies

Single Dater: I want to use this app the keep my apartment clean so I can impress the people I am dating.

College Students: I want to keep my place clean so I can have a clear head when doing college coursework.

Anyone wanting to stay organized: I want my place to be clean so I can complete other tasks as they arise.

Usage Context

This will be a mobile and computer based application, that will help keep personal cleaning done so as not to get embarrassed when dating or when friends come over. The user will need internet connectivity and a smartphone/computer to use the app.

User Stories being used for the Cleaning Buddy App

1. As a Cleaning Buddy user, I want to create an account so that I can save my cleaning preferences and history.

This user story is being used because it is necessary to initiate a relationship between the user and the app.

2. As a Cleaning Buddy user, I want to log in so that I can access my saved cleaning schedules and settings.

This user story is being used because it allows the user to store their login information so future use can happen.

3. As a Cleaning Buddy user, I want to add rooms and areas so that tasks can be assigned to specific locations.

This user story is being used because different users may have different dwelling types.

4. As a Cleaning Buddy user, I want to set task frequency (daily/weekly/monthly/**yearly**) so that chores repeat at desired intervals.

This user story is being used so the user can organize their cleaning.

5. As a Cleaning Buddy user, I want to set task priority so that important chores are highlighted and completed first. **1=urgent, 2=important, 3=Luxury**

6. This user story is being used so the user can set priorities to more important chores.

7. As a Cleaning Buddy user, I want to create checklists for tasks so that steps are clear and consistent.

This user story is being used because checklists are totally inclusive and leaves nothing out.

8. As a Cleaning Buddy user, I want to mark tasks complete so that my progress is recorded.

This user story is being used so that the user will know what has been completed and what needs to be done.

9. As a Cleaning Buddy user, I want to set due dates and reminders so that tasks are completed on time.

This user story is being used so the user can be organized throughout the week with other tasks other than cleaning.

10. As a Cleaning Buddy user, I want an overview dashboard so that I can quickly see pending, overdue, and completed tasks.

This user story is being used so that the user can see at a quick glance what is completed and what is overdue.

11. As a Cleaning Buddy user, I want calendar integration (**Google Calendar**) so that cleaning tasks appear in my primary calendar.

This user story is being used so that cleaning can be schedule with other weekly tasks.

12. As a Cleaning Buddy user, I want cleaning tips and best practices per task so that chores are done effectively. **He doesn't want the tips to be intrusive, wants AI to pull cleaning tips per room that pop up and can be dismissed easily.**

This user story is being used to give the user best practices to cleaning before they start cleaning.

13. As a Cleaning Buddy user, I want localization (language/time zone) support so that the app fits my region and schedule.

This user story is being used so that the user will have the correct date and time when they schedule cleanings.

14. As a Cleaning Buddy user, I want offline access to my current checklist so that I can work without internet and sync later.

This user story is being used because sometimes the internet is not working properly and we do not want to waste time by not having access to our cleaning tasks.

15. As a Cleaning Buddy user, I want analytics on task completion trends so that I can identify problem areas or busy periods.

This user story is being used so users can analyze why cleaning certain areas are a problem and to help with improvement. A simple analytics page

16. As a Cleaning Buddy user, I want in-app help and FAQs so that I can quickly solve common issues without contacting support.

This user story is being used so that the user will not have to waste time contacting support for app troubleshooting.

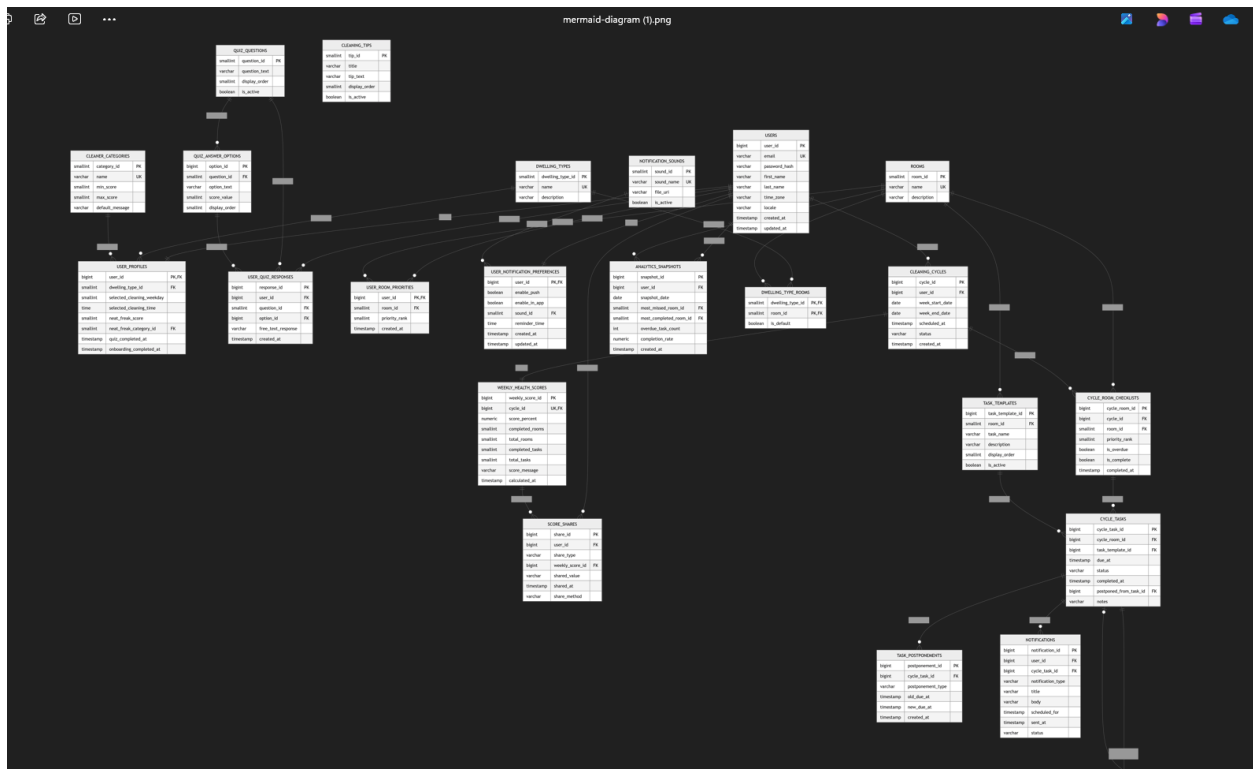
17. As a Cleaning Buddy user, I want to reset my password so that I can regain access if I forget it.

18. This user story is being used because users do forget their passwords and need to regain access to their accounts.

19. As a Cleaning Buddy user, I want push notifications for upcoming tasks so that I don't forget scheduled chores.

This user story is being used to remind the user they have chores to complete

ERD and Schema for the Cleaning Buddy



Key:

- ||--|| = exactly 1 to exactly 1
- ||--o{ = 1 to 0..many
- ||--o| = 1 to 0..1
- o|--o{ or self-reference labels can be used for optional parent-child relationships

One correction from the earlier version: user_profiles should usually relate to dwelling_types and cleaner_categories as **many profiles to one category/type**, but Mermaid ERD expresses that from the parent side as DWELLING_TYPES ||--o{ USER_PROFILES and CLEANER_CATEGORIES ||--o{ USER_PROFILES, which is what's shown here.

Please see the ERD for all Primary and Foreign Keys, Listed below are SQL tables

CREATE TABLE cleaner_categories (

```
category_id SMALLSERIAL PRIMARY KEY,  
name VARCHAR(50) NOT NULL UNIQUE,  
min_score SMALLINT NOT NULL CHECK (min_score BETWEEN 0 AND 100),  
max_score SMALLINT NOT NULL CHECK (max_score BETWEEN 0 AND 100),  
default_message VARCHAR(255) NOT NULL,  
CHECK (min_score <= max_score)  
);
```

```
CREATE TABLE dwelling_types (  
dwelling_type_id SMALLSERIAL PRIMARY KEY,  
name VARCHAR(50) NOT NULL UNIQUE,  
description VARCHAR(255)  
);
```

```
CREATE TABLE rooms (  
room_id SMALLSERIAL PRIMARY KEY,  
name VARCHAR(50) NOT NULL UNIQUE,  
description VARCHAR(255)  
);
```

```
CREATE TABLE users (  
user_id BIGSERIAL PRIMARY KEY,  
email VARCHAR(255) NOT NULL UNIQUE,  
password_hash VARCHAR(255) NOT NULL,  
first_name VARCHAR(100),  
last_name VARCHAR(100),  
time_zone VARCHAR(100) DEFAULT 'UTC',  
locale VARCHAR(20) DEFAULT 'en-US',  
created_at TIMESTAMPT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMPT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE user_profiles (  
user_id BIGINT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,  
dwelling_type_id SMALLINT NOT NULL REFERENCES dwelling_types(dwelling_type_id),  
selected_cleaning_weekday SMALLINT NOT NULL CHECK (selected_cleaning_weekday  
BETWEEN 0 AND 6),  
selected_cleaning_time TIME NOT NULL,  
neat_freak_score SMALLINT CHECK (neat_freak_score BETWEEN 0 AND 100),  
neat_freak_category_id SMALLINT REFERENCES cleaner_categories(category_id),  
quiz_completed_at TIMESTAMPT,  
onboarding_completed_at TIMESTAMPT,  
created_at TIMESTAMPT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
updated_at TIMESTAMPT NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```
);
```

```
CREATE TABLE dwelling_type_rooms (  
    dwelling_type_id SMALLINT NOT NULL REFERENCES dwelling_types(dwelling_type_id) ON  
DELETE CASCADE,  
    room_id SMALLINT NOT NULL REFERENCES rooms(room_id) ON DELETE CASCADE,  
    is_default BOOLEAN NOT NULL DEFAULT TRUE,  
    PRIMARY KEY (dwelling_type_id, room_id)  
);
```

```
CREATE TABLE quiz_questions (  
    question_id SMALLSERIAL PRIMARY KEY,  
    question_text VARCHAR(500) NOT NULL,  
    display_order SMALLINT NOT NULL,  
    is_active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
CREATE TABLE quiz_answer_options (  
    option_id BIGSERIAL PRIMARY KEY,  
    question_id SMALLINT NOT NULL REFERENCES quiz_questions(question_id) ON DELETE  
CASCADE,  
    option_text VARCHAR(255) NOT NULL,  
    score_value SMALLINT NOT NULL,  
    display_order SMALLINT NOT NULL  
);
```

```
CREATE TABLE user_quiz_responses (  
    response_id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
    question_id SMALLINT NOT NULL REFERENCES quiz_questions(question_id),  
    option_id BIGINT REFERENCES quiz_answer_options(option_id),  
    free_text_response VARCHAR(500),  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (user_id, question_id)  
);
```

```
CREATE TABLE user_room_priorities (  
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
    room_id SMALLINT NOT NULL REFERENCES rooms(room_id),  
    priority_rank SMALLINT NOT NULL CHECK (priority_rank IN (1, 2)),  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (user_id, priority_rank),  
    UNIQUE (user_id, room_id)  
);
```



```

CREATE TABLE task_templates (
  task_template_id BIGSERIAL PRIMARY KEY,
  room_id SMALLINT NOT NULL REFERENCES rooms(room_id),
  task_name VARCHAR(150) NOT NULL,
  description VARCHAR(255),
  display_order SMALLINT NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  UNIQUE (room_id, task_name)
);

```

```

CREATE TABLE cleaning_cycles (
  cycle_id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  week_start_date DATE NOT NULL,
  week_end_date DATE NOT NULL,
  scheduled_at TIMESTAMP NOT NULL,
  status VARCHAR(20) NOT NULL CHECK (status IN
('scheduled','in_progress','completed','partially_completed','missed')),
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (user_id, week_start_date)
);

```

```

CREATE TABLE cycle_room_checklists (
  cycle_room_id BIGSERIAL PRIMARY KEY,
  cycle_id BIGINT NOT NULL REFERENCES cleaning_cycles(cycle_id) ON DELETE CASCADE,
  room_id SMALLINT NOT NULL REFERENCES rooms(room_id),
  priority_rank SMALLINT CHECK (priority_rank IN (1, 2) OR priority_rank IS NULL),
  is_overdue BOOLEAN NOT NULL DEFAULT FALSE,
  is_complete BOOLEAN NOT NULL DEFAULT FALSE,
  completed_at TIMESTAMP,
  UNIQUE (cycle_id, room_id)
);

```

```

CREATE TABLE cycle_tasks (
  cycle_task_id BIGSERIAL PRIMARY KEY,
  cycle_room_id BIGINT NOT NULL REFERENCES cycle_room_checklists(cycle_room_id) ON
DELETE CASCADE,
  task_template_id BIGINT NOT NULL REFERENCES task_templates(task_template_id),
  due_at TIMESTAMP NOT NULL,
  status VARCHAR(20) NOT NULL CHECK (status IN
('pending','completed','postponed','overdue')),
  completed_at TIMESTAMP,
  postponed_from_task_id BIGINT REFERENCES cycle_tasks(cycle_task_id),

```

```
    notes VARCHAR(500),  
    UNIQUE (cycle_room_id, task_template_id)  
);
```

```
CREATE TABLE task_postponements (  
    postponement_id BIGSERIAL PRIMARY KEY,  
    cycle_task_id BIGINT NOT NULL REFERENCES cycle_tasks(cycle_task_id) ON DELETE CASCADE,  
    postponement_type VARCHAR(20) NOT NULL CHECK (postponement_type IN  
('tomorrow','next_week')),  
    old_due_at TIMESTAMP NOT NULL,  
    new_due_at TIMESTAMP NOT NULL,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE notification_sounds (  
    sound_id SMALLSERIAL PRIMARY KEY,  
    sound_name VARCHAR(50) NOT NULL UNIQUE,  
    file_uri VARCHAR(255),  
    is_active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
CREATE TABLE user_notification_preferences (  
    user_id BIGINT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,  
    enable_push BOOLEAN NOT NULL DEFAULT TRUE,  
    enable_in_app BOOLEAN NOT NULL DEFAULT TRUE,  
    sound_id SMALLINT REFERENCES notification_sounds(sound_id),  
    reminder_time TIME,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE notifications (  
    notification_id BIGSERIAL PRIMARY KEY,  
    user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
    cycle_task_id BIGINT REFERENCES cycle_tasks(cycle_task_id) ON DELETE SET NULL,  
    notification_type VARCHAR(20) NOT NULL CHECK (notification_type IN ('push','in_app')),  
    title VARCHAR(150) NOT NULL,  
    body VARCHAR(500) NOT NULL,  
    scheduled_for TIMESTAMP NOT NULL,  
    sent_at TIMESTAMP,  
    status VARCHAR(20) NOT NULL CHECK (status IN ('pending','sent','failed','dismissed'))  
);
```

```
CREATE TABLE weekly_health_scores (  
    -->
```

```

weekly_score_id BIGSERIAL PRIMARY KEY,
cycle_id BIGINT NOT NULL UNIQUE REFERENCES cleaning_cycles(cycle_id) ON DELETE
CASCADE,
score_percent NUMERIC(5,2) NOT NULL CHECK (score_percent BETWEEN 0 AND 100),
completed_rooms SMALLINT NOT NULL DEFAULT 0,
total_rooms SMALLINT NOT NULL DEFAULT 0,
completed_tasks SMALLINT NOT NULL DEFAULT 0,
total_tasks SMALLINT NOT NULL DEFAULT 0,
score_message VARCHAR(255) NOT NULL,
calculated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE score_shares (
share_id BIGSERIAL PRIMARY KEY,
user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
share_type VARCHAR(20) NOT NULL CHECK (share_type IN ('neat_freak','weekly_health')),
weekly_score_id BIGINT REFERENCES weekly_health_scores(weekly_score_id) ON DELETE
SET NULL,
shared_value VARCHAR(100) NOT NULL,
shared_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
share_method VARCHAR(30) NOT NULL CHECK (share_method IN ('clipboard','manual'))
);

CREATE TABLE cleaning_tips (
tip_id SMALLSERIAL PRIMARY KEY,
title VARCHAR(100) NOT NULL,
tip_text VARCHAR(255) NOT NULL,
display_order SMALLINT NOT NULL,
is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE analytics_snapshots (
snapshot_id BIGSERIAL PRIMARY KEY,
user_id BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
snapshot_date DATE NOT NULL,
most_missed_room_id SMALLINT REFERENCES rooms(room_id),
most_completed_room_id SMALLINT REFERENCES rooms(room_id),
overdue_task_count INTEGER NOT NULL DEFAULT 0,
completion_rate NUMERIC(5,2) NOT NULL CHECK (completion_rate BETWEEN 0 AND 100),
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
UNIQUE (user_id, snapshot_date)
);

```

Cleaning Buddy Data Modeling Narrative and Business Rules Overview

The Cleaning Buddy application is designed to help users manage and maintain household cleaning tasks through structured scheduling, reminders, and performance tracking. The data model supports a **single-user system** where each user configures their cleaning preferences, completes a one-time quiz, selects a weekly cleaning day, and tracks progress through room-based task completion.

The database is designed using a **relational model** to ensure data integrity, reduce redundancy, and support scalability for features such as analytics, notifications, and progress tracking.

Design Goals

The data model was designed with the following goals:

1. **Support onboarding and personalization**
 - One-time cleanliness quiz
 - Dwelling type selection
 - Top two room priorities
 2. **Enable structured task management**
 - Predefined room-based cleaning tasks
 - Checklist-based workflow (no custom tasks)
 3. **Support scheduling constraints**
 - One weekly cleaning day
 - Limited postponement options (tomorrow or next week)
 4. **Track progress and performance**
 - Room completion logic (all tasks must be complete)
 - Weekly cleanliness health score (0–100%)
 5. **Provide notifications and insights**
 - Push and in-app notifications
 - Cleaning tips and analytics
-

Core Entities and Their Roles

1. Users and Profiles

- **Users** represent individuals using the app.
- **User_Profiles** store configuration data collected during onboarding.

Each user:

- Has exactly **one profile**
- Selects a **dwelling type**
- Chooses **one cleaning day**
- Receives a **neat freak score and category**

This separation ensures authentication data is isolated from user preferences.

2. Cleaner Categories

The **Cleaner_Categories** entity defines the five classification levels:

- Minimalist Maintainer
- Casual Cleaner
- Routine Ready
- Neat Freak

- Spotless Specialist

Each category corresponds to a score range (0–100).

This allows:

- Consistent classification
 - Reusable logic for messaging and analytics
-

3. Quiz System

The onboarding quiz is modeled using:

- **Quiz_Questions**
- **Quiz_Answer_Options**
- **User_Quiz_Responses**

This structure supports:

- A **one-time quiz per user**
- Scored answers for calculating the neat freak score
- Flexibility to modify questions without altering schema

Each user can only answer each question once, enforcing the “one-time quiz” requirement.

4. Dwelling and Room Structure

The app organizes cleaning by location:

- **Dwelling_Types** (apartment, studio, house)
- **Rooms** (kitchen, bathroom, bedroom, etc.)
- **Dwelling_Type_Rooms** (mapping table)

This allows:

- Different room configurations per dwelling type
 - Scalability if new dwelling types are added
-

5. Task Templates

Cleaning tasks are predefined and stored in:

- **Task_Templates**

Each task:

- Belongs to a room
- Is reused across all users

This design enforces:

- Standardized cleaning checklists
 - No custom task creation (as required)
-

6. User Priorities

The app allows users to select **two priority rooms**, stored in:

- **User_Room_Priorities**

Constraints:

- Only **two priorities allowed**
- Ranked as **1 (highest)** and **2**

These priorities influence:

- Scheduling

- Notifications
 - Analytics
-

7. Scheduling and Cleaning Cycles

Cleaning is managed weekly using:

- **Cleaning_Cycles**

Each cycle represents:

- One week of cleaning
- One scheduled cleaning day

This ensures:

- Only **one cleaning day per week**
 - Easy tracking of weekly performance
-

8. Room Checklists and Task Instances

To track progress:

- **Cycle_Room_Checklists** represent each room in a cycle
- **Cycle_Tasks** represent individual task instances

Important rule:

- A **room is only complete if all tasks are complete**

This structure allows:

- Accurate room-level completion tracking
 - Task-level progress monitoring
-

9. Postponements (**May need to re-evaluate after this update**)

Postponement behavior is restricted using:

- **Task_Postponements**

Constraints:

- Only two options:
 - "Tomorrow"
 - "Next week"
 - **Wants to be able to input a date**

This ensures:

- Controlled scheduling
 - Simplicity in logic / **May change the logic due to date input, possible calendar pop-up**
-

10. Notifications

The notification system includes:

- **Notification_Sounds**
- **User_Notification_Preferences**
- **Notifications**

Users can:

- Enable push or in-app reminders
- Select a notification sound

The system logs all notifications sent for tracking and debugging.

11. Weekly Health Scores

User performance is summarized in:

- **Weekly_Health_Scores**

This table calculates:

- Percentage of completed tasks/rooms
- Score between 0–100%
- Associated feedback message

Each cleaning cycle produces exactly **one health score**.

12. Sharing and Analytics

Additional support features include:

- **Score_Shares**
 - Allows users to copy/share scores (no social media integration)
- **Analytics_Snapshots**
 - Stores aggregated performance data
 - Identifies problem areas (e.g., most missed room)

13. Cleaning Tips

- **Cleaning_Tips** provides static best practices such as:
 - Clean top to bottom
 - Dust before sweeping

This table supports:

- Simple content delivery
- Easy updates without code changes

Key Relationships and Cardinality

Some important relationships include:

- One user → one profile (**1:1**)
- One user → many cleaning cycles (**1:M**)
- One cycle → many rooms (**1:M**)
- One room → many tasks (**1:M**)
- One cycle → one health score (**1:1**)

These relationships enforce:

- Data consistency
- Logical flow of cleaning activities

Constraints and Business Rules

The data model enforces key application rules:

- Only **one quiz attempt per user**
- Only **one cleaning day per week**
- Only **two priority rooms**
- Only **two postponement options**
- Tasks must be completed to mark a room complete

- The app is **single-user only** (no shared households)
-

Normalization and Integrity

The schema is normalized to approximately **Third Normal Form (3NF)**:

- No repeating groups
- No partial dependencies
- No transitive dependencies

Benefits:

- Reduced redundancy
 - Improved consistency
 - Easier maintenance
-

Conclusion

The Cleaning Buddy data model is designed to align closely with the app's functional requirements while maintaining a clean, scalable, and normalized structure. It supports the full lifecycle of the user experience:

1. **Onboarding → Quiz → Profile setup**
2. **Task generation → Scheduling → Notifications**
3. **Task completion → Room completion → Weekly scoring**
4. **Insights → Analytics → Sharing**

This structure ensures that the application remains simple for users while being robust enough to support future enhancements.